- *Relational data*—Elasticsearch is not the right tool to accommodate search over data that has relationships and needs to perform complex database joins. Elasticsearch is not designed to handle complex relational data structures. If your data is relationship-heavy, a relational database like MySQL or PostgreSQL may be a better fit. Most modern databases (MySQL, PostgreSQL, etc.) also offer full-text search capabilities, although the features are not as advanced as in a modern search engine like Elasticsearch.
- *Transactional data*—Elasticsearch is an "eventually consistent" search engine, which makes it unsuitable for applications that require immediate consistency, such as financial transactions. For these types of use cases, consider using a traditional relational database or a NoSQL database like MongoDB.
- *Geospatial data*—While Elasticsearch has built-in support for geospatial data, it may not be the most efficient solution for large-scale geospatial analytics. For these use cases, consider using a dedicated geospatial database like PostGIS or a geospatial analytics platform like ArcGIS.
- *High-write workloads*—Elasticsearch can handle high-read workloads, but it is not optimized for high-write workloads. If you need to index large amounts of data in real time, consider using a dedicated indexing engine like Apache Flume or Apache Kafka.
- *Online analytical processing (OLAP) data*—If you need to perform complex multidimensional analysis on large data sets, a traditional OLAP database like Microsoft Analysis Services or IBM Cognos may be a better fit than Elasticsearch.
- *Large binary data*—While Elasticsearch can handle large amounts of text data, it may not be the best solution for indexing and searching large binary data like videos or images. For these use cases, consider using a dedicated binary data store like Hadoop Distributed File System (HDFS), Amazon S3, or Azure Files.
- *Real-time analytics*—Elasticsearch is great for performing real-time search and analytics on large data sets, but it may not be the most efficient solution for real-time data processing and analytics. Instead, consider a specialized real-time analytics platform like Apache Spark or Apache Flink.
- *Latency-sensitive applications*—Although Elasticsearch is designed to handle high-volume search and analytical queries, it can still have latency issues when dealing with large amounts of data. For applications that require sub-millisecond response times, a specialized search engine like Apache Solr or a columnar database like Apache Cassandra may be a better fit.
- *Other types*—Elasticsearch is not a preferred solution for time-series data, graph data, in-memory data, and various other types of data. If you need to store and analyze time-series data, a specialized time-series database like InfluxDB or TimescaleDB may be a better fit. Similarly, a graph database such as Neo4j may help you tackle graph data.

It's important to evaluate your specific use case and requirements before choosing Elasticsearch as your technology and tool. In the next section, we discuss common misconceptions about Elasticsearch as a tool, a technology, and a search solution.